

Week 1 - Day 1a: Operations and Data Types

You will be working on a series of exercises for the material covered in lecture 1a. As a reminder, we discussed programming languages and how Python/Jupyter can be used as a powerful calculator.

The first set of exercises will focus on how to use Python/Jupyter as a calculator.

Useful Tips:

- Jupyter consists of a series of cells
- Use shift + enter to run a cell
- Remember the order of operations!

Exercise 1: Operations []

Calculate the following quantities *using Jupyter*. Use *shift + enter* to run cells

1.1 `10 + 20`

```
# TODO: Your code here
```

1.2 `72 - 123`

```
# TODO: Your code here
```

1.3 `23 * 67`

```
# TODO: Your code here
```

1.4 `90 / 30`

```
# TODO: Your code here
```

1.5 `90 / 27`

```
# TODO: Your code here
```

Exercise 2: Exponents []

Beyond addition, subtraction, multiplication, and division, there are other, more complex operations.

The *syntax* for exponentiation is as `a ** b`, not `^`!

Calculate these operations below on a piece of paper then verify your answer using Jupyter below.

2.1 `10 ** 5`

TODO: *Your code here*

2.2 `25 ** 0.5`

TODO: *Your code here*

2.3 `3 ** -1`

TODO: *Your code here*

Exercise 3: Floor Operations and Modulus []

The syntax for floor division, also called integer division, is `a // b`. It gives the whole number answer to division. The syntax for modulus is `a % b`. Modulus calculates the remainder for division. For example:

`9 / 4 = 2.25`
`9 // 4 = 2`
`9 % 4 = 1`

Remember to use *shift + enter* to run cells

3.1 `11 // 5`

TODO: *Your answer here*

3.2 `11 % 5`

TODO: *Your answer here*

3.3 `16 // 2`

TODO: *Your answer here*

3.4 `1 % 2`

TODO: *Your answer here*

3.5 `8 % 4`

TODO: *Your answer here*

```
3.6 (67 % 10) // 3
```

```
# TODO: Your answer here
```

Exercise 4: Errors [] []

In Python, there are some *invalid operations*. Invalid operations will cause errors. If you see an error, that means that there's a logical or syntactical issue with your code. These will become increasingly important later in the course to help you debug your code.

Explain why each of the following are incorrect and write the error type.

4.1 `10 / 0`

4.2 `1 +`

4.3 `2 * * 4`

4.4 `3 x 4`

4.5 `3 a`

Exercise 5: Operations on Different Data Types [] []

A `float` is a number with a decimal. An `int` is an integer. However, despite `1.0` being a whole number, it has a decimal. Therefore, `1.0` is a float. For examples:

```
type(1)          = int
type(10000)       = int
type(3.14)        = float
type(2.0)         = float
```

Note: Types are very important. Later today, we will see a function called `str` that converts any data into strings.

5.1 What is the datatype of `5`? How about `5.0`?

Operations will change the datatype of the answer.

Here are the Rules:

- Division always gives a float
- Floor division always gives an integer
- Addition, subtraction, multiplication, and exponentiation will always give a float if one or both of the operands are a float

For example:

- `3 * 1 = 3`
- `3.0 * 1.0 = 3.0`
- `1 + 1.0 = 2.0`

- $3 / 3 = 1$.

Write the answers to each of the following. If the answer results in a float, be sure to put the decimal!

5.2 `3.0 - 2`

5.3 `2 // 4.0`

5.4 `6 / 2`

5.5 `6.0 / 2.0`

5.6 `2**2`

5.7 `2.0**2`

5.8 `2**2.0`

Exercise 6: Order of Operations []]

Now that you're used to simple arithmetic, we'll make things a bit harder. Python/Jupyter executes statements like `1 + 2 * 3` in a specific order. This is called the *order of operations*.

The order of operation is very important! They can change the results dramatically. Note, the order of operations also affects the data type.

Compute the following quantities by hand. Be sure to put a decimal if the data type is a float.

6.1 `1 + 2 * 3`

6.2 `1 * 2 + 3`

6.3 `1 / 2 + 3`

6.4 `1 - 2 * 3`

6.5 `1.0 * 2 - 3`

Exercise 7: Function Output []]

Now that we've gone over the basic arithmetic operations, let's use them in interesting ways.

Let's say we have a mathematical function $f(x) = x^3 + 2x^2 + 3x + 4$.

Important Note: we will see later in the course that *programming function* are **NOT THE SAME** as *mathematical functions*. You don't need to worry about this for now, but keep this in mind for **Week 1 Day 3!**

7.1 Compute $f(5)$. Expand the computation so that x is replaced by 5 and you do all of the arithmetic operations.

TODO: *Your code here*

7.2 Now, compute $f(10)$.

You might notice that this process of substituting the variables is quite annoying. We will revisit this in future lectures.

```
# TODO: Your code here
```

Exercise 8: Extracting Digits [□□□]

For this exercise, using *only the operations we discussed in class*, do the following:

8.1 Extract the right-most digit from the number 48756378291, i.e. the digit 1.

```
# TODO: Your code here
```

8.2 Extract the second right-most digit from the number 48756378291, i.e. the digit 9.

```
# TODO: Your code here
```

8.3 Extract the third right-most digit from the number 48756378291, i.e. the digit 2.

```
# TODO: Your code here
```

8.4 What is the pattern in the previous three parts? Explain in natural language.

If you've completed all of the exercises so far, congratulations! There are supplementary exercises you can do now. Ask a TA to get access to them.